

Цель работы: изучить операторы, используемые при обработке исключительных ситуаций, возникающих во время выполнения вычислительных процессов, получить практические навыки в составлении программ.

Выполнение заданий:

Вариант 11

Задание 1: Смоделировать структуру предприятия:

Классы	Свойства и методы
Фирма	название (get, set)
Отдел	название (get, set) количество сотрудников (get, set)
Сотрудник	фио (get, set) должность (get, set) оклад (get, set) рассчитать зарплату()
Штатный сотрудник	премия (get, set) рассчитать зарплату()
Сотрудник по контракту	рассчитать зарплату()

Обработать все Exception с помощью блока try...catch(Exception ...) в методе «рассчитать зарплату» классов Штатный сотрудник и Сотрудник по контракту. При возникновении Exception выводить на экран сообщение об ошибке.

Описать собственный класс Exception PremiyaException. В методе «рассчитать зарплату» класса Штатный сотрудник выбрасывать собственное Exception типа PremiyaException при отрицательном значении свойства Премия. В этом же методе обработать PremiyaException в блоке catch. При возникновении Exception выводить сообщение об ошибке на экран.

В основной программе проверить работу блока обработки Exception метода «рассчитать зарплату».

Описать собственный класс Exception OkladException. В конструкторе реализовать проверку значения оклада, при отрицательном значении выбрасывать собственное Exception типа OkladException. Обработать Exception OkladException с помощью блока try...catch, в блоке обработки Exception вывести на экран сообщение «Невозможно создать сотрудника – указан отрицательный оклад: <оклад> » и повторно создать Exception.

В основной программе (main) обработать вызов конструктора класса Сотрудник и проверить работу обработчика Exception.

Листинг или ссылка на проект:

```
using System;

namespace MultiSidedSolution
{
    class Firm
```

```

{
    string title;

    public Firm(string title)
    {
        this.title = title;
    }
}
class Department
{
    string title;
    int employeeCount;

    public Department(string title, int employeeCount)
    {
        this.title = title;
        this.employeeCount = employeeCount;
    }
}
class Employee
{
    protected string fio;
    protected string position;
    protected double salary;

    public Employee(string fio, string position, double salary)
    {
        this.fio = fio;
        this.position = position;
        this.salary = salary;
        try
        {
            if (salary < 0)
                throw new OkladException($"Невозможно создать сотрудника - указан отрицательный оклад: {salary}");
        }
        catch (OkladException oe) { Console.ForegroundColor = ConsoleColor.DarkRed; Console.WriteLine(oe.Message); Console.ResetColor(); }
    }
    public virtual double CalculateSalary(int monthLen)
    {
        Random r = new Random();
        try { return Math.Round((salary / monthLen) * r.Next(15, monthLen - 8 + 1), 2); }
        catch (Exception) { Console.Write("[Произошла ошибка]"); }
        return 0;
    }
    /*
     * ЗПП = О / КМ x РД
     * ЗПП – заработная плата к получению;
     * О – оклад или тарифная ставка;
     * КМ – длительность календарного месяца (28,29,30 или 31 день);
     * РД – количество отработанных рабочих дней в календарном месяце;
     */
    public override string ToString() => $"{fio} \t{position} \t${salary}";
}
class StaffMember : Employee
{
    double award;
    public StaffMember(string fio, string position, double salary, double award) : base(fio, position, salary)
    {
        this.award = award;
    }
    public override double CalculateSalary(int monthLen)
    {
        try
        {
            if (award < 0)
                throw new PremiyaException("[Премия меньше нуля]");
        }
    }
}

```

```

        return Math.Round(base.CalculateSalary(monthLen) + award, 2);
    }
    catch (PremiyaException pe) { Console.ForegroundColor = ConsoleColor.Magenta;
Console.Write(pe.Message); Console.ResetColor(); }
    return 0;
}

    public override string ToString() => $"{fio}    \t{position}    \t{salary}    \t{award}";
}
class ContractEmployee : Employee
{
    public ContractEmployee(string fio, string position, double salary) : base(fio, position,
salary) { }
}

class PremiyaException : Exception
{
    public PremiyaException(string message) : base(message) { }
}
class OkladException : Exception
{
    public OkladException(string message) : base(message) { }
}
class Program
{
    static string StrGenerator(int x)
    {
        Random r = new Random();
        string s = "";
        for (int i = 0; i < x; i++)
            s += (char)r.Next(65, 81);
        return s;
    }
    static void Main()
    {
        Random r = new Random();
        int monthLen = r.Next(28, 32);

        Employee[] empoloyes = new Employee[50];
        for (int i = 0; i < empoloyes.Length / 2; i++)
            empoloyes[i] = new StaffMember(StrGenerator(r.Next(5, 11)),
StrGenerator(r.Next(5, 11)), Math.Round(r.Next(-1000, 9001) + r.NextDouble(), 2),
Math.Round(r.Next(-1000, 5001) + r.NextDouble(), 2));
        for (int i = empoloyes.Length / 2; i < empoloyes.Length; i++)
            empoloyes[i] = new ContractEmployee(StrGenerator(r.Next(5, 11)),
StrGenerator(r.Next(5, 11)), Math.Round(r.Next(-1000, 9001) + r.NextDouble(), 2));
        for (int i = 0; i < empoloyes.Length; i++)
        {
            Console.Write(empoloyes[i].ToString() + " ");
            Console.ForegroundColor = ConsoleColor.Green;
            Console.WriteLine(" $" + empoloyes[i].CalculateSalary(monthLen));
            Console.ResetColor();
        }
    }
}
}

```

Результат работы программы (копия экрана):

```
[Невозможно создать сотрудника - указан отрицательный оклад: -904,54]
[Невозможно создать сотрудника - указан отрицательный оклад: -739,24]
[Невозможно создать сотрудника - указан отрицательный оклад: -606,42]
[Невозможно создать сотрудника - указан отрицательный оклад: -868,44]
[Невозможно создать сотрудника - указан отрицательный оклад: -319,93]
[Невозможно создать сотрудника - указан отрицательный оклад: -371,3]
DEPHACH      IDLDPMPIJMH      5519,55      1134,39      $4446,12
JBMPD        HHFPDFM          3944,43      3060,87      $5427,53
HKILM        KHCHO            1584,92      -562,44      [Премия меньше нуля] $0
FBOIA        FMMLABIC         -904,54      4999,61      $4426,73
MCOGEIPNF    OLMINLE          7107,47      3653,72      $7918,2
EODOG        HGCEIOMK         -739,24      1922,52      $1405,05
ADGAOEJMK    NKAGPI           5462,08      1357,18      $4816,5
BMPHFDPBK    EMDCMAJMP        2996,27      -409,76      [Премия меньше нуля] $0
BJJHFFIKPD   FHOIOGGOL        2489,31      1490,86      $3067,42
ILKEMEPCG    LPIGJPCPBK       7614,04      4040,91      $9624,54
BAALGGKM     MILNAHFB         5693,78      628,02      $3664,7
EPHFADCMA    FCIKEB           4894,73      482,82      $2930,18
ODIELHDIDP   BDFGE            202,82      3290,25      $3391,66
BABEAA       AHFCAJN          2471,57      2731,09      $4543,57
PBJIECC      JJBNEACCKO       1405,16      -785,46      [Премия меньше нуля] $0
ONFIFJCG     HNMKKGAK         3132,74      2003,73      $4092,22
CHMIILHM     EIFEDBNMHB       3823,38      1543,79      $3582,93
EGHGMLLC     FDOMO            3424,53      1417,36      $3243,78
EGGHLINAD    JHIOEJBGKK       1299,62      3956,4       $4692,85
CACMAJFJ     OOLEM            -606,42      1454,43      $1151,22
LAPHPHKOP    LFBMPBNEA        1717,05      -565,39      [Премия меньше нуля] $0
MNEKGFK      DIPOJD           4521,39      857,75      $4173,44
ADMCEJOKC    MCFPHGIBDJ       -868,44      4557,51      $4065,39
DPJCBAL      CLBGEDAGIP        3392,88      1518,13      $3780,05
CAFOGC       MMONA            7111,01      38,49      $4542,13
ANFJJCOAG    MMAEDNIBMA       $4594,05      $2297,02
IADBLOJH     DMHKAECAL        $4192,6       $2655,31
GDKHF        CJACEDAPP         $227,23       $113,62
PBIENEKNP    PLANCCM           $4932,48      $2466,24
EBJDLGHIJF   BEONKMAGKG        $2899,01      $1739,41
EEMMC        PHEPDCCJ          $8399,57      $6159,68
CHOLPHEBEC   NFJBDALCKA        $1656,14      $1214,5
LLDIKLBDP    IODNO             $6990,4       $4660,27
AMCED        HHMFN             $5961,38      $4371,68
EBFIGF       HGNEKJNNL         $1367,27      $820,36
MDJEE        DONBEF            $3699,81      $2589,87
PEIFBFOLL    FKLJDJ            $-319,93      $-181,29
LMFHNAHI     BHKIKOLOM         $952,82       $603,45
LIDLMKJP     MELCP             $3146,16      $1573,08
ALIACAOPBL   NNPNNHKOPJ        $-371,3       $-259,91
JAINGGP      KBMGOF           $7651,01      $5355,71
KEMMN        OBHDHMELO         $1261,75      $925,28
EIINOGBJ     NOBNFE            $1961,14      $1372,8
KKEBCBEKM    KHBHL             $8699,75      $5219,85
JCIFB        OKDML             $7538,78      $5528,44
BPBAEHEDNO   PHCCMO            $4989,91      $2993,95
ADLDLMIFB    MJAHJGDKEP        $6762,84      $3606,85
FHKBNIGM     HFHAPNFM          $2854,2       $1617,38
FKPJCHK      KEHCGNNM          $7018,66      $4445,15
DKIOBAN      DCEOJMG           $6778,99      $4971,26
```

Контрольные вопросы

1. Что такое «исключительная ситуация (исключение)»?

Исключение – то, что в условиях программы произойти не может

2. Для чего предназначен механизм исключений в языке C#?

Механизм исключений (Exception), предусмотренный в C#, упрощает обработку ошибок. Вместо того чтобы проверять значение, возвращаемое функциями и методами, вы можете использовать для обнаружения и обработки ошибок структурные операторы, такие как try и catch.

3. Какие операторы используются для обработки исключений?

Класс **Exception**, ключевые слова **try**, **catch**, **throw**

4. Для чего используется ключевое слово try?

Для обозначения области кода, которую нужно проверять на исключения

5. Проиллюстрируйте обработку исключительной ситуации фрагментом программы на языке C#.

```
try {  
    // блок кода }  
catch (ТипException1) {  
    // обработчик Exception типа ТипException1 }  
catch (ТипException2) {  
    // обработчик Exception типа ТипException2  
    throw(e)    // повторное возбуждение Exception }  
finally {  
}
```

6. Требуется ли соблюдение соответствия типа перехватываемого исключения типу исключения в обработчике catch?

Да

7. Как реализуется перехват всех исключений?

`catch(Exception){}`

8. Что происходит, если исключение не обработано?

Программа прекращает выполнение с оповещением об ошибке

9. Как выполняется оператор catch без параметров?

```
catch  
{  
    /*Обработка ошибки*/  
}
```

10. Объясните назначение свойства StackTrace класса System.Exception.

Свойство StackTrace позволяет определить последовательность вызовов, которая привела к возникновению исключения.

11. Может ли быть в программе блок try без блока catch?

Нет

12. Куда передается управление после обработки исключительной ситуации?

В блок finally. Если его нет, то далее по коду

Выводы: я изучил операторы, используемые при обработке исключительных ситуаций, возникающих во время выполнения вычислительных процессов, получил практические навыки в составлении программ.